

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
customer_data= pd.read_csv('/customer behaviour analysis.csv')
```

```
customer_data.head ()
```

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Rating
0	8411	15-09-2023 12:24	Home	298	3	2340	Credit Card	4.5
1	2103	15-09-2023 11:58	Electronics	286	4	3561	Credit Card	4.0
2	36353	15-09-2023 11:55	Books	288	1	3983	Cash	4.5
3	9806	15-09-2023 11:52	Clothing	329	2	1519	PayPal	4.0
4	14198	15-09-2023 11:50	Books	95	3	792	PayPal	4.5

```
from datetime import date
import pandas as pd # Ensure pandas is imported

#finding out each cx rfm values

#recency

snapshot_date = pd.to_datetime('2023-09-16')

customer_data['Purchase Date'] = pd.to_datetime(customer_data['Purchase Date'])

# Calculate Recency for each customer.
# Group by 'Customer ID' and find the difference in days
# between the snapshot_date and the customer's last purchase date.
recency = customer_data.groupby(['Customer ID']).agg({
    'Purchase Date': lambda x: (snapshot_date - x.max()).days
})

# Rename the column for clarity
```

```
recency.rename(columns={'Purchase Date': 'Recency'}, inplace=True)
recency
```

Recency 	
Customer ID	
1	58
2	298
3	89
4	126
5	171
...	...
49996	271
49997	49
49998	11
49999	420
50000	127

49673 rows × 1 columns

Next steps:

[Generate code with recency](#)[New interactive sheet](#)

```
freq_df = customer_data.drop_duplicates(subset=["Customer ID", "Purchase Dat
freq = freq_df.groupby(["Customer ID"]).size().reset_index(name='Frequency')
```

```
freq.head()
```

1 to 5 of 5 entries Filter  		
index	Customer ID	Frequency
0	1	1
1	2	3
2	3	8
3	4	4
4	5	8

Show 25 per pageLike what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps:

[Generate code with freq](#)[New interactive sheet](#)

#monetary

```
customer_data["Total Purchase Amount"] - customer_data["Product Price"] * cu
```

	0
0	1446
1	2417
2	3695
3	861
4	507
...	...
249995	1524
249996	1993
249997	504
249998	1983
249999	2461

250000 rows × 1 columns

dtype: int64

```
monetary_value = customer_data.groupby("Customer ID")["Total Purchase Amount"]  
monetary_value
```

	Total Purchase Amount
Customer ID	
1	3491
2	7988
3	22587
4	8715
5	12524
...	...
49996	14044
49997	22431
49998	8610
49999	6984
50000	4116

49673 rows × 1 columns

dtype: int64

```
rfm_df = pd.concat([recency, freq, monetary_value], axis=1)
rfm_df.head()
```

	Recency	Customer ID	Frequency	Total Purchase Amount	
1	58.0	2.0	3.0	3491.0	
2	298.0	3.0	8.0	7988.0	
3	89.0	4.0	4.0	22587.0	
4	126.0	5.0	8.0	8715.0	
5	171.0	6.0	6.0	12524.0	

Next steps:

[Generate code with rfm_df](#)[New interactive sheet](#)

```
rfm_df.columns
```

```
Index(['Recency', 'Customer ID', 'Frequency', 'Total Purchase Amount'],
      dtype='object')
```

```
spend_cols = ('Recency', 'Customer ID', 'Frequency', 'Total Purchase Amount')
```

```
rfm_df["total_spending"] = rfm_df[list(spend_cols)].sum(axis=1)
rfm_df.head()
```

	Recency	Customer ID	Frequency	Total Purchase Amount	total_spending
1	58.0	2.0	3.0	3491.0	3554.0
2	298.0	3.0	8.0	7988.0	8297.0
3	89.0	4.0	4.0	22587.0	22684.0
4	126.0	5.0	8.0	8715.0	8854.0
5	171.0	6.0	6.0	12524.0	12707.0

Next steps:

[Generate code with rfm_df](#)[New interactive sheet](#)

```
rfm_df["total_spending"]
```

	total_spending
1	3554.0
2	8297.0
3	22684.0
4	8854.0
5	12707.0
...	...
48852	49179.0
48941	49275.0
49126	49458.0
49153	49486.0
49314	49646.0

50000 rows × 1 columns

dtype: float64

```
customer_first_purchase_dates = customer_data.groupby('Customer ID')['Purchase Date']
customer_first_purchase_dates.rename(columns={'Purchase Date': 'First_Purchase_Date'})
rfm_df = pd.merge(rfm_df, customer_first_purchase_dates[['Customer ID', 'First_Purchase_Date']], on='Customer ID')
today = pd.to_datetime('today')
rfm_df["customer_since"] = (today - rfm_df["First_Purchase_Date"]).dt.days
rfm_df.drop(columns=['First_Purchase_Date'], inplace=True)
rfm_df.head()
```

	Recency	Customer ID	Frequency	Total Purchase Amount	total_spending	customer_since
0	58.0	2.0	3.0	3491.0	3554.0	1947.0
1	298.0	3.0	8.0	7988.0	8297.0	2105.0
2	89.0	4.0	4.0	22587.0	22684.0	2094.0
3	126.0	5.0	8.0	8715.0	8854.0	2349.0
4	171.0	6.0	6.0	12524.0	12707.0	2357.0

Next steps:

[Generate code with rfm_df](#)

[New interactive sheet](#)

```
def segment(row):
    if row['Total Purchase Amount'] > 3000:
        return "High Value"
    elif row['Total Purchase Amount'] > 1000:
```

```

        return "Medium Value"
    else:
        return "Low Value"

rfm_df['Segment'] = rfm_df.apply(segment, axis=1)
rfm_df.head()

```

	Recency	Customer ID	Frequency	Total Purchase Amount	total_spending	customer_since	Segment
0	58.0	2.0	3.0	3491.0	3554.0	1947.0	
1	298.0	3.0	8.0	7988.0	8297.0	2105.0	
2	89.0	4.0	4.0	22587.0	22684.0	2094.0	
3	126.0	5.0	8.0	8715.0	8854.0	2349.0	
4	171.0	6.0	6.0	12524.0	12707.0	2357.0	

Next steps:

[Generate code with rfm_df](#)[New interactive sheet](#)

```

def customer_segment(row):
    if row['Recency'] <= 171 and row['Frequency'] >= 6.0 and row['Total Purchase Amount'] >= 12500:
        return 'Best Customers'
    elif row['Recency'] <= 60 and row['Frequency'] >= 3:
        return 'Loyal Customers'
    elif row['Recency'] <= 30 and row['Frequency'] == 1:
        return 'New Customers'
    elif row['Recency'] > 60 and row['Frequency'] >= 3:
        return 'At Risk Customers'
    else:
        return 'Lost Customers'

rfm_df['Segment'] = rfm_df.apply(customer_segment, axis=1)

```

```
rfm_df['Segment'].value_counts()
```

	count
Segment	
At Risk Customers	31401
Lost Customers	6387
Loyal Customers	6185
Best Customers	5848
New Customers	179

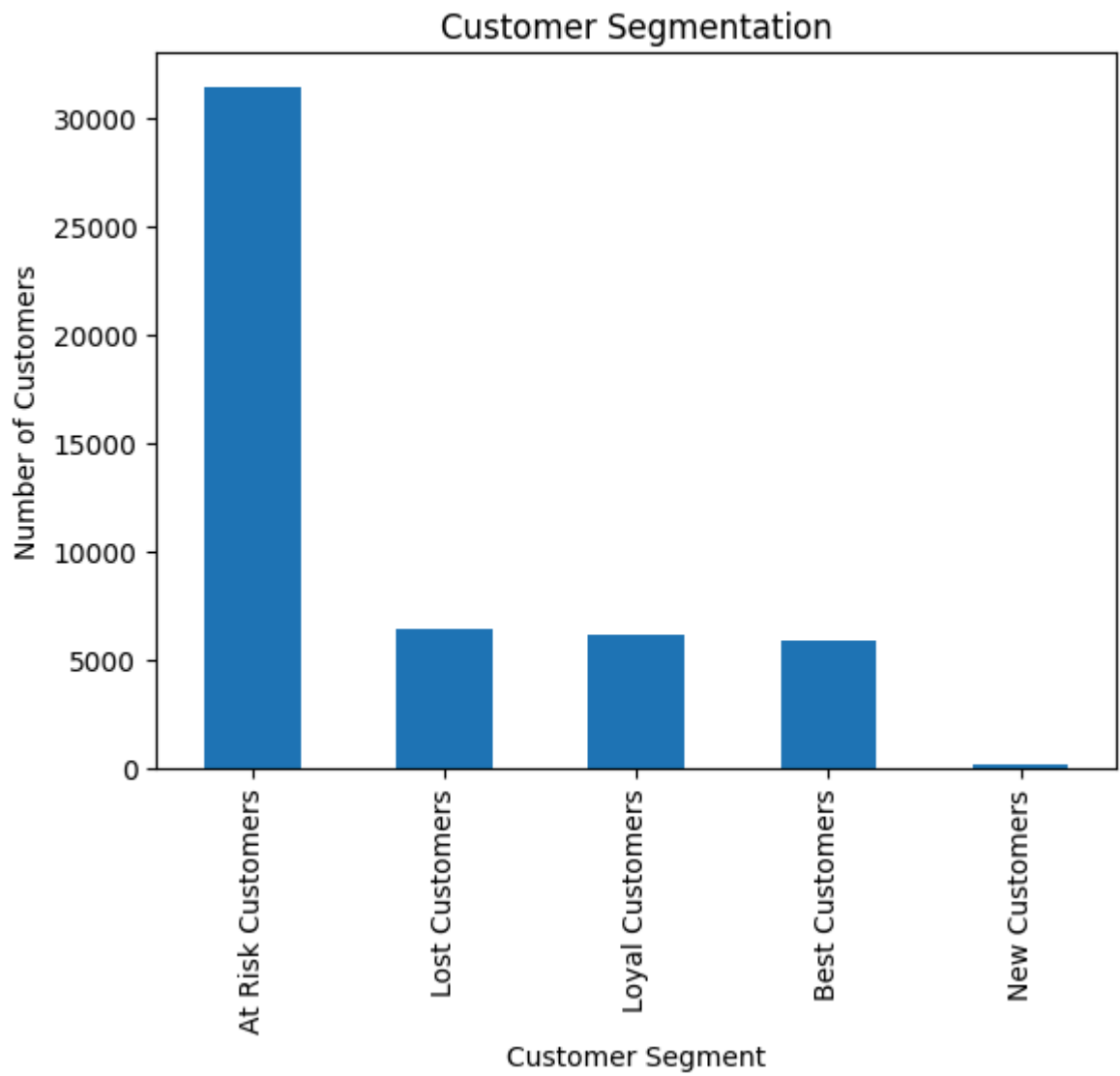
dtype: int64

```
segment_count = rfm_df['Segment'].value_counts()  
segment_count
```

	count
Segment	
At Risk Customers	31401
Lost Customers	6387
Loyal Customers	6185
Best Customers	5848
New Customers	179

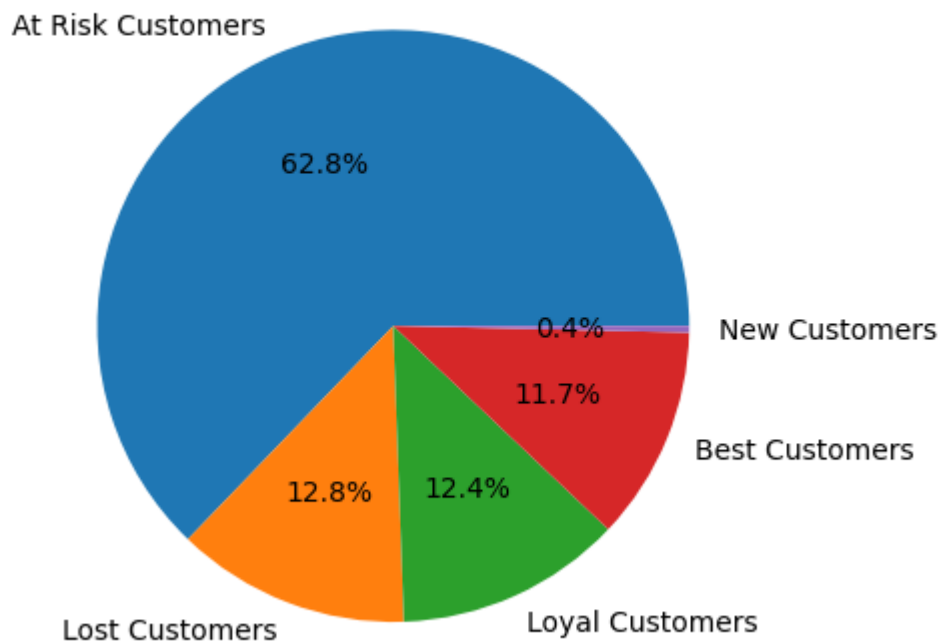
dtype: int64

```
import matplotlib.pyplot as plt  
  
segment_count.plot(kind='bar')  
  
plt.title('Customer Segmentation')  
plt.xlabel('Customer Segment')  
plt.ylabel('Number of Customers')  
plt.show()
```



```
segment_count.plot(kind='pie', autopct='%1.1f%%')  
  
plt.title('Customer Segmentation')  
plt.ylabel('')  
plt.show()
```

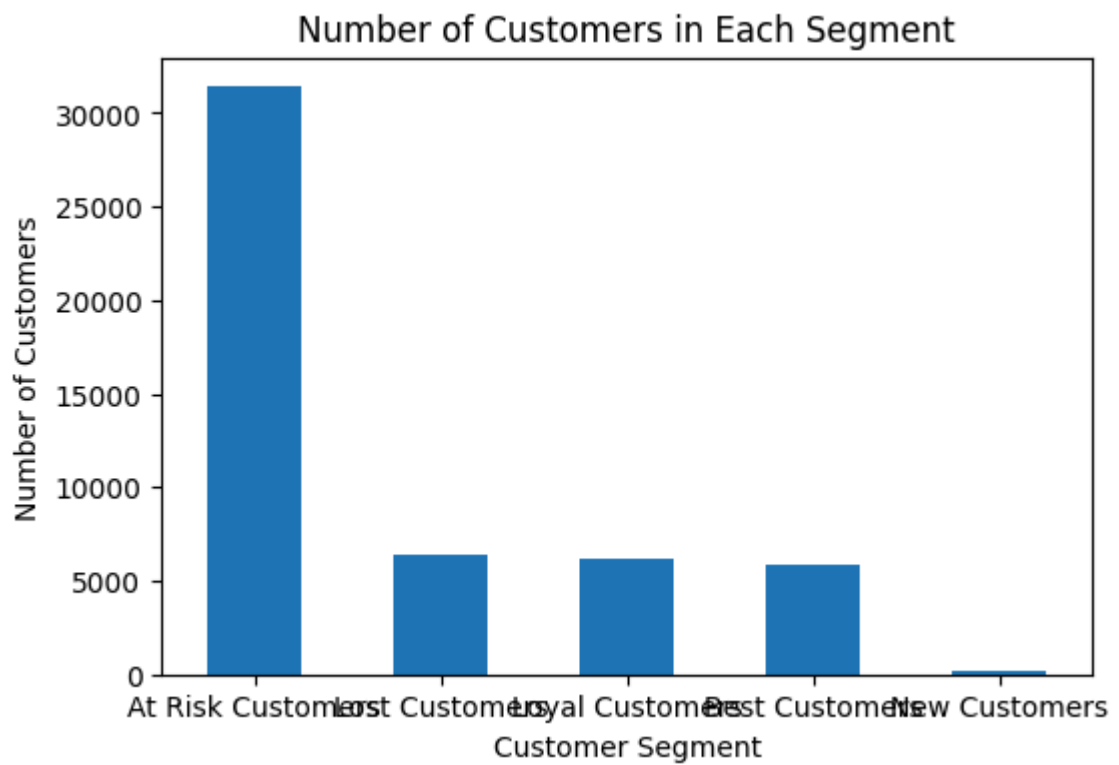

Customer Segmentation



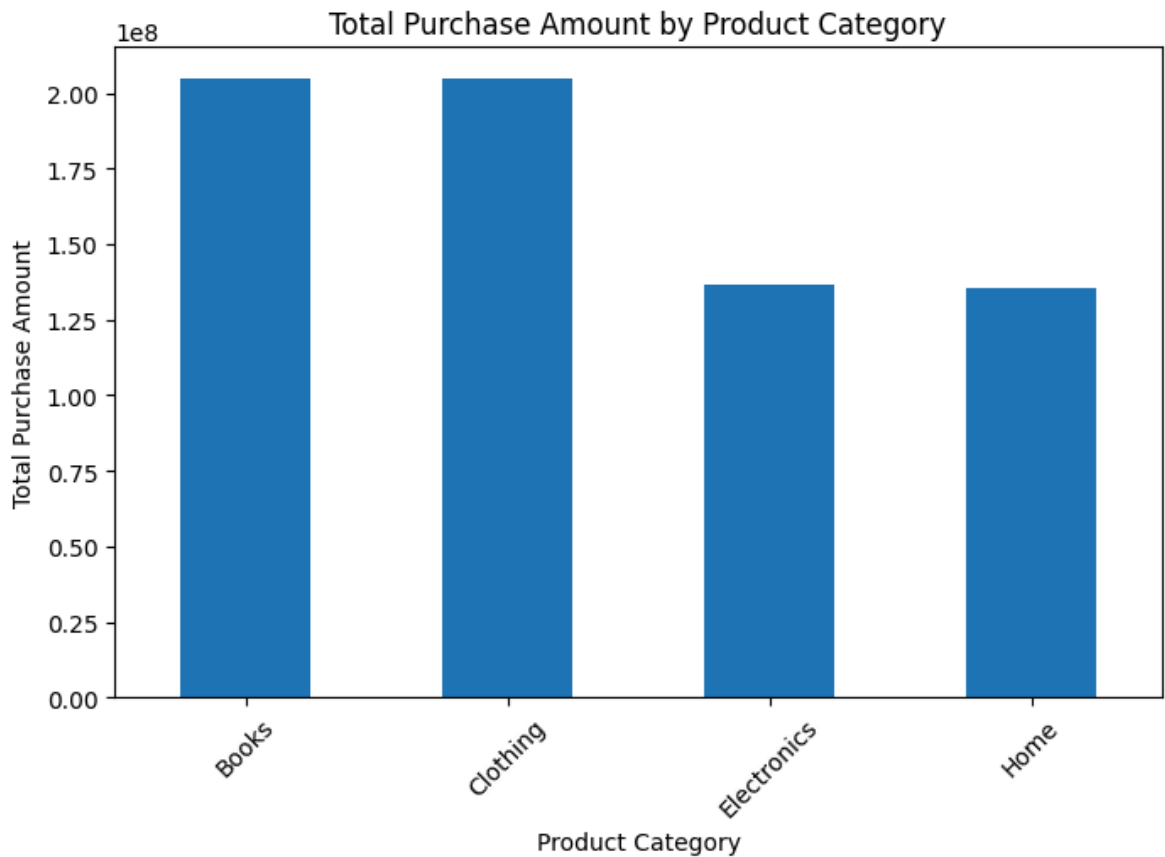
```
import matplotlib.pyplot as plt

segment_count = rfm_df['Segment'].value_counts()

plt.figure(figsize=(6,4))
segment_count.plot(kind='bar')
plt.title('Number of Customers in Each Segment')
plt.xlabel('Customer Segment')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.show()
```



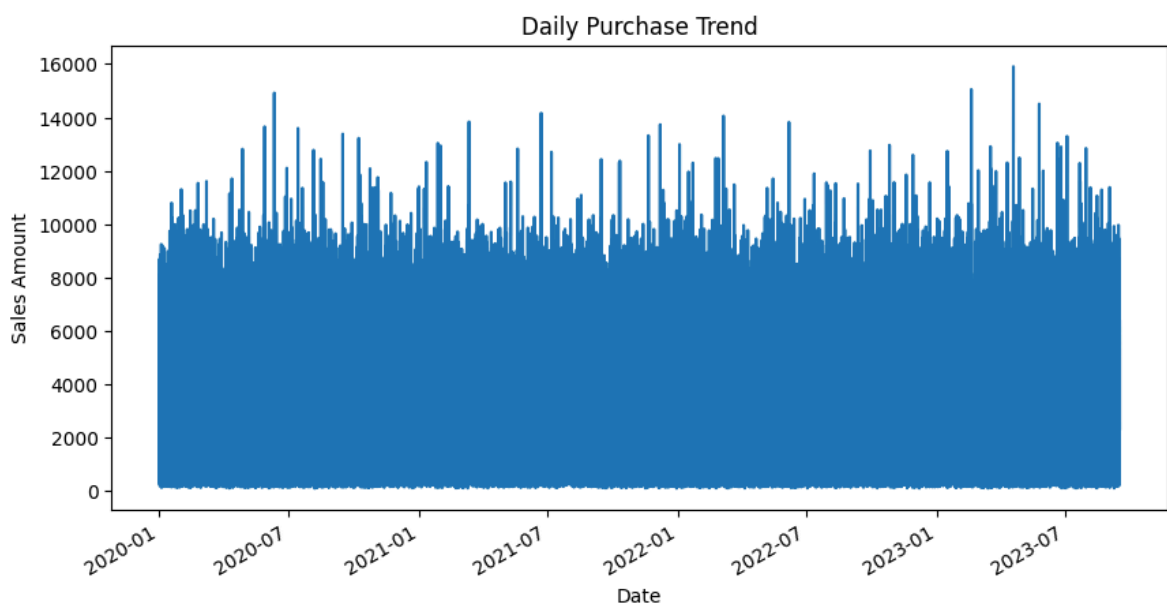
```
category_sales = customer_data.groupby('Product Category')['Total Purchase A  
  
plt.figure(figsize=(8,5))  
category_sales.plot(kind='bar')  
plt.title('Total Purchase Amount by Product Category')  
plt.xlabel('Product Category')  
plt.ylabel('Total Purchase Amount')  
plt.xticks(rotation=45)  
plt.show()
```



```
customer_data['Purchase Date'] = pd.to_datetime(customer_data['Purchase Date'])

daily_sales = customer_data.groupby('Purchase Date')['Total Purchase Amount']

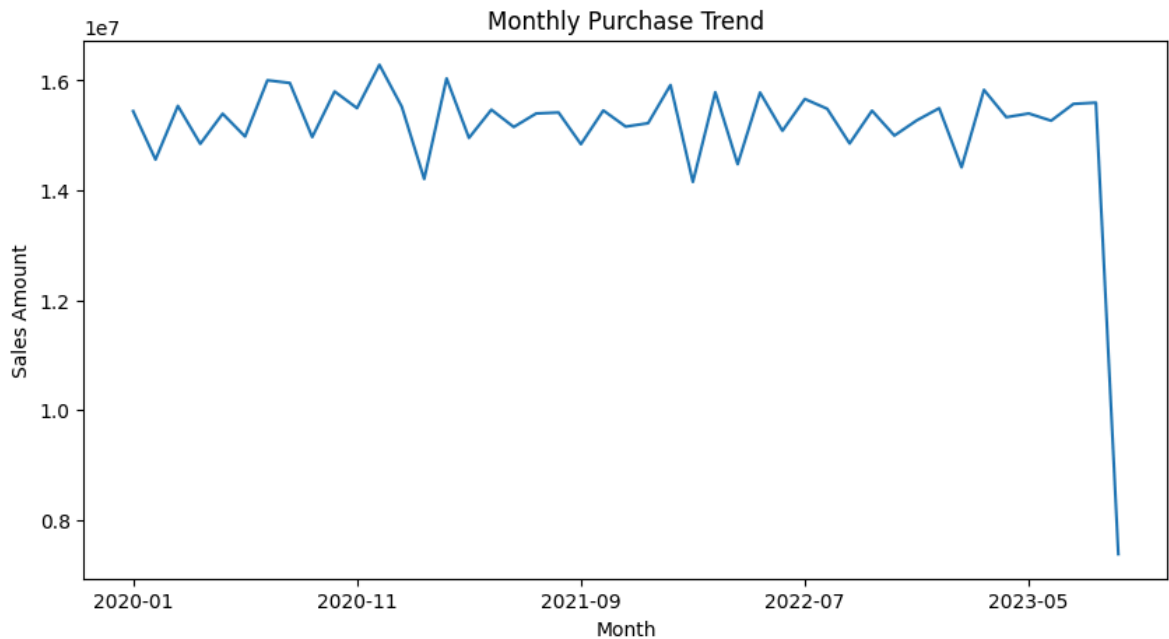
plt.figure(figsize=(10,5))
daily_sales.plot(kind='line')
plt.title('Daily Purchase Trend')
plt.xlabel('Date')
plt.ylabel('Sales Amount')
plt.show()
```



```
monthly_sales = customer_data.groupby(customer_data['Purchase Date'].dt.to_p

monthly_sales.index = monthly_sales.index.astype(str)

plt.figure(figsize=(10,5))
monthly_sales.plot(kind='line')
plt.title('Monthly Purchase Trend')
plt.xlabel('Month')
plt.ylabel('Sales Amount')
plt.show()
```



```
top_customers = rfm_df.groupby('Customer ID')['Total Purchase Amount'].sum

plt.figure(figsize=(10,5))
top_customers.plot(kind='bar')
plt.title('Top 10 Customers by Purchase Amount')
plt.xlabel('Customer ID')
plt.ylabel('Total Purchase Amount')
plt.xticks(rotation=45)
plt.show()
```

